

Введение

В современной индустрии видеоигр наблюдается значительный рост числа независимых разработчиков (indie-dev), которым требуется специализированная платформа для продвижения своих продуктов, коммуникации с аудиторией и организации технической поддержки. Существующие решения (Steam, Epic Games Store, itch.io) фокусируются преимущественно на дистрибуции, оставляя нишу комплексных инструментов поддержки и взаимодействия с пользователями практически незанятой.

Цель дипломного проекта заключается в разработке и реализации веб-платформы, объединяющей функционал каталога игр, системы подписок, технической поддержки через тикеты, публикации контента (новости, гайды) и социального взаимодействия (чат, комментарии, подписки на разработчиков).

На дипломный проект определены следующие задачи:

- Провести анализ предметной области и сформулировать функциональные требования.
- Спроектировать реляционную базу данных с учётом сложных связей между сущностями.
- Разработать серверную часть на базе Next.js App Router с REST API.
- Реализовать клиентский интерфейс с разграничением доступа по ролям (User, Developer, Admin).
- Обеспечить механизмы аутентификации, авторизации и защиты данных.
- Провести функциональное тестирование разработанной системы.

Объектом исследования являются процессы поддержки и продвижения программных продуктов (видеоигр) в цифровой среде.

					ДП.09.02.07.25.18 ПЗ	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

Предметом исследования является архитектура, алгоритмы и программные средства информационной системы для поддержки indie-разработчиков.

					ДП.09.02.07.25.18 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

1. Анализ предметной области и обзор существующих решений для продвижения и поддержки игр

1.1. Описание предметной области

Рынок цифровой дистрибуции видеоигр характеризуется высокой конкуренцией. По данным Steam, ежегодно публикуется более 10 000 новых игр, что создаёт проблему «видимости» для независимых разработчиков.

Ключевые проблемы indie-разработчиков:

- Отсутствие прямой коммуникации с аудиторией — крупные платформы ограничивают возможности обратной связи.
- Сложность организации технической поддержки — отсутствие интегрированных инструментов трекинга багов.
- Монетизация раннего доступа — необходимость инструментов подписок и краудфандинга.
- Необходимость публикации контента — новости, гайды, обновления требуют отдельных каналов.

Анализ существующих решений показал отсутствие комплексной платформы, объединяющей каталог, поддержку, подписки и социальные функции в едином интерфейсе.

1.2 Анализ программных аналогов данной предметной области

Саб-реддит , посвящённый ассетам для гейм разработки. Плюсы — комьюнити, можно найти единомышленников и получить независимую оценку на свои арты. Или найти себе начинающего художника в игру и ничего не искать и не рисовать. Из минусов ресурса — нет нормального поиска. Магазин, есть раздел с free-наборами персонажей и фонов, еще есть звуки, но выбор бесплатных ресурсов не очень большой.

					ДП.09.02.07.25.18 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

Itch.io — любимая многими платформа с уникальной моделью "плати сколько хочешь". Разработчики сами устанавливают процент комиссии (от 0% до 30%). Itch.io особенно ценится за простой процесс публикации, поддержку jam-проектов и лояльное сообщество, готовое поддержать экспериментальные игры. Из минусов — засорение платформы, некоторые пользователи отмечают, что на Itch.io много игр, которые занимают больше времени на загрузку, чем на игру. Также есть проблема с играми, которые требуют больше времени на загрузку, чем на прохождение.

Game Jolt — платформа, ориентированная на инди-сообщество с сильным социальным компонентом. Комиссия составляет 10%, что делает её привлекательной для начинающих разработчиков. Game Jolt позволяет монетизировать игры через рекламу, что может быть интересной альтернативой для бесплатных проектов. Минус платформы — ограничения для разработчиков, например, запрет на использование некоммерческих изображений, музыки и кода на платформе. Также на Game Jolt нельзя продавать игры.

Indie DB — хотя в первую очередь это сообщество и база данных инди-игр, платформа также предлагает возможности для дистрибуции. Особенно полезна для наработки аудитории на ранних стадиях разработки. Минус платформы — устаревший интерфейс, некоторые пользователи отмечают, что дизайн платформы выглядит устаревшим.

Kongregate — хотя эта платформа в основном фокусируется на браузерных играх, она может быть отличным местом для размещения HTML5-версий Godot-проектов. Платформа предлагает деление доходов от рекламы.

Все эти платформы часто предлагают возможности, которые трудно найти в крупных магазинах: прямой контакт с сообществом, более гибкие условия и меньшую конкуренцию. Они могут стать отличным стартовым пунктом для новых разработчиков или тестовой площадкой для экспериментальных идей. Но у всех ранее перечисленных платформ есть свои

					ДП.09.02.07.25.18 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

недостатки и в данном проекте соединены все вышеперечисленные платформы, исправлены их минусы и созданы в одну платформу.

1.2. Формирование требований к проектируемой информационной системе

Формирование функциональных требований является главной составляющей для платформы, для того чтобы определить ее дальнейшие перспективы и направления.

В таблице 1 представлены функциональные требования

Таблица 1 – Функциональные требования

Номер	Требование
1	Регистрация и аутентификация пользователей
2	Публикация игр с медиа-контентом (обложки, скриншоты, файлы)
3	Просмотр каталога игр с фильтрацией по жанрам и платформам
4	Система подписок на игры (tier-модель: Bronze/Silver/Gold)
5	Создание и обработка тикетов (bug/feature/question)
6	Публикация новостей и гайдов
7	Комментирование статей с поддержкой вложенных ответов
8	Личные сообщения между пользователями
9	Подписка на разработчиков (Follow)
10	Управление пользователями и ролями
11	Система уведомлений о событиях
12	Оценка и отзывы игр (1-5 звёзд)

1.3. Обоснование выбора технологического стека

Выбор фреймворка: Next.js 15 (App Router)

Next.js выбран по следующим причинам:

- App Router — современная модель роутинга с серверными компонентами, позволяющая выполнять запросы к БД на стороне сервера без клиентского JavaScript.
- Fullstack в одном проекте — единая кодовая база для frontend и backend (API Routes).
- SSR/SSG/ISR — гибкие стратегии рендеринга для оптимизации производительности.
- Экосистема Vercel — оптимизированный деплой и CI/CD.

Выбор ORM: Prisma 5.22

Prisma обеспечивает:

- Type-safe запросы — автогенерация типов на основе схемы.
- Миграции — версионирование структуры БД через prisma migrate.
- Prisma Studio — визуальный инструмент для работы с данными (порт 5555).
- Поддержка SQLite для dev-режима с возможностью миграции на PostgreSQL.

Выбор аутентификации: iron-session

В отличие от JWT, iron-session:

- Шифрует данные сессии в cookie (Seal), не требуя хранения состояния на сервере.
- Не подвержен проблеме «утечки секретного ключа» JWT (токены нельзя отозвать).
- Поддерживает secure и httpOnly флаги cookie.

Выбор стилизации: TailwindCSS

Утилитарный CSS-фреймворк обеспечивает:

- Консистентный дизайн через конфигурацию темы.
- Минимальный размер CSS-бандла (purge unused styles).

					ДП.09.02.07.25.18 ПЗ	Лист
						9
Изм.	Лист	№ докум.	Подпись	Дата		

- Быструю прототипирование интерфейсов.

2. Проектирование архитектуры системы и разработка ER-модели базы данных

2.1. Разработка ER-модели базы данных для платформы

База данных спроектирована в виде реляционной модели с использованием Prisma Schema. Включает 20+ сущностей. В информационной системе база данных, отраженная в ERD, играет главную роль, как хранилище данных о платформе: данные игр, пользователей, статей, подписок, хранящихся файлов и остальном.

ER-диаграмма базы данных информационной системы представлен на рисунке 2.1.

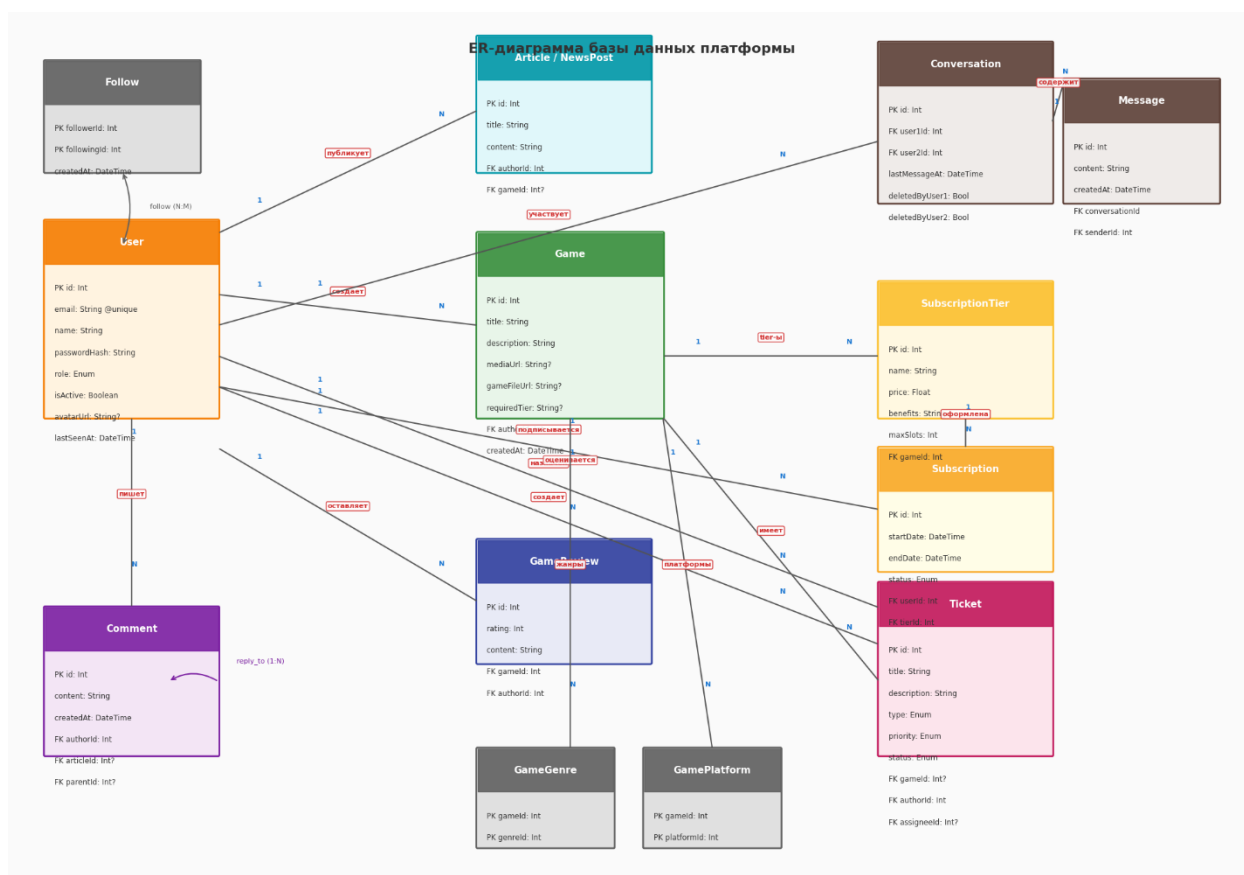


Рисунок 2.1. - ER-диаграмма базы данных

Уникальные ограничения

- GameReview: @unique([gameId, authorId]) — один отзыв на игру от пользователя.
- Conversation: @unique([user1Id, user2Id]) — единый чат между двумя пользователями.

Стратегии onDelete

- Cascade — для зависимого контента (медиа, отзывы, тикеты при удалении игры; сообщения при удалении диалога).
- SetNull — для необязательных связей (статья может потерять привязку к игре).

2.2 Назначение и область применения

Текст пж

Функциональные возможности разрабатываемой платформы представлены в диаграмме вариантов использования на рисунке 2.2.

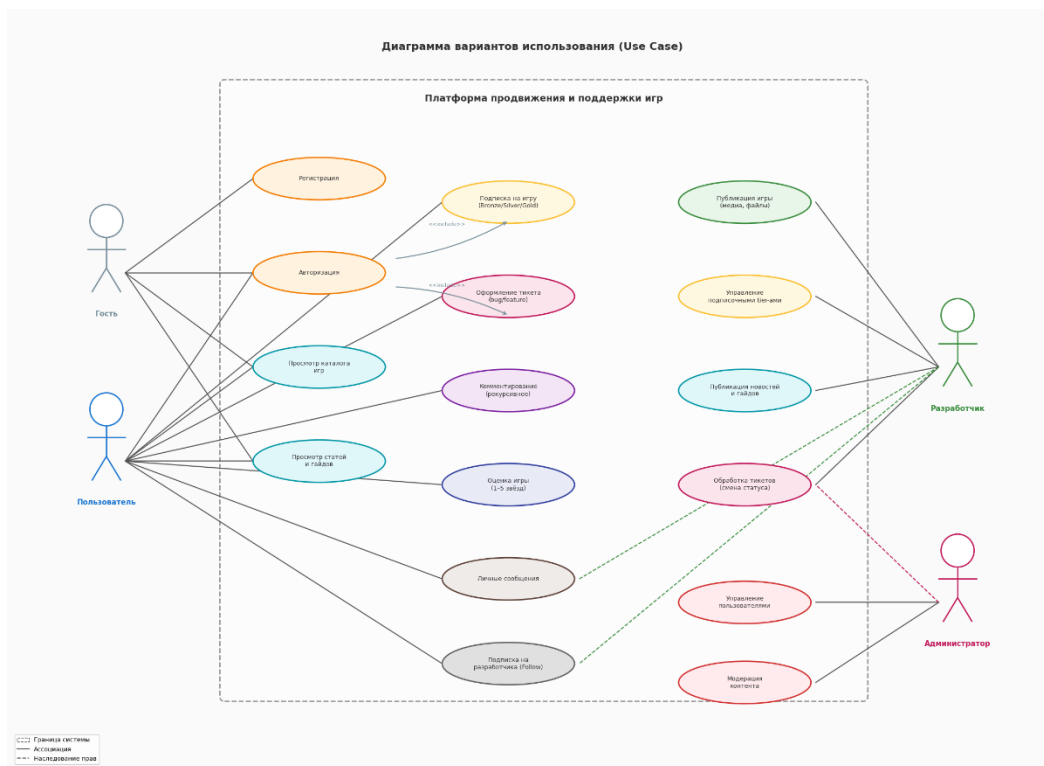


Рисунок 2.2. - Диаграмма вариантов использования

3. Программная реализация клиентской и серверной частей платформы.

3.1. Проектирование серверной части (Backend)

Архитектура API

Серверная часть реализована через Next.js App Router API Routes (src/app/api/.../route.ts). Иерархия включает 30+ эндпоинтов:

Аутентификация:

- Метод POST в /api/auth/register — регистрация с хешированием bcrypt.

Код аутентификации изображен на рисунке 3.1.

					ДП.09.02.07.25.18 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		12

```

export async function POST(request: Request) {
  try {
    const { email, name, password, role, age } = await request.json();

    const existingUser = await prisma.user.findUnique({ where: { email } });
    if (existingUser) {
      return NextResponse.json({ error: 'Email уже используется' }, { status: 400 });
    }

    const passwordHash = await bcrypt.hash(password, 10);
    const user = await prisma.user.create({
      data: {
        email,
        name,
        passwordHash,
        role: role || 'user',
        age: age ? Number(age) : null,
      },
    });

    const session = await getSession();
    session.userId = user.id;
    session.email = user.email;
    session.name = user.name;
    session.role = user.role;
    session.age = user.age ?? undefined;
    await session.save();

    return NextResponse.json({ user }, { status: 201 });
  } catch (error) {
    console.error(error);
    return NextResponse.json({ error: 'Ошибка сервера' }, { status: 500 });
  }
}

```

Рисунок 3.1. – Код аутентификации

– Метод POST в /api/auth/login — создание сессии iron-session.

Код создания сессии изображен на рисунке 3.2.

```

export async function POST(request: Request) {
  try {
    const { email, password } = await request.json();

    const user = await prisma.user.findUnique({ where: { email } });
    if (!user || !user.passwordHash) {
      return NextResponse.json({ error: 'Неверный email или пароль' }, { status: 401 });
    }

    const isValid = await bcrypt.compare(password, user.passwordHash);
    if (!isValid) {
      return NextResponse.json({ error: 'Неверный email или пароль' }, { status: 401 });
    }

    if (!user.isActive) {
      return NextResponse.json({ error: 'Ваш аккаунт деактивирован' }, { status: 403 });
    }

    const session = await getSession();
    session.userId = user.id;
    session.email = user.email;
    session.name = user.name;
    session.role = user.role;
    session.age = user.age ?? undefined;
    await session.save();

    return NextResponse.json({ user });
  } catch (error) {
    console.error(error);
    return NextResponse.json({ error: 'Ошибка сервера' }, { status: 500 });
  }
}

```

Рисунок 3.2. – Код создания сессии

– Метод POST в /api/auth/logout — удаление сессии.

Код удаления сессии изображен на рисунке 3.3.

```

export async function POST() {
  const session = await getSession();
  session.destroy();
  return NextResponse.json({ ok: true });
}

```

Рисунок 3.3. – Код удаления сессии

- Метод GET в /api/auth/me — получение текущего пользователя.
- Код получения текущего пользователя изображен на рисунке 3.4.

```
export async function GET() {
  try {
    const session = await getSession();
    if (!session.userId) {
      return NextResponse.json({ user: null });
    }

    const user = await prisma.user.findUnique({
      where: { id: session.userId },
      select: {
        id: true,
        email: true,
        name: true,
        role: true,
        avatarUrl: true,
        showFollowers: true,
      },
    });

    if (!user) {
      return NextResponse.json({ user: null });
    }

    return NextResponse.json({
      user: {
        userId: user.id,
        email: user.email,
        name: user.name,
        role: user.role,
        avatarUrl: user.avatarUrl,
        showFollowers: user.showFollowers,
      },
    });
  } catch (error) {
    console.error('Ошибка в /api/auth/me:', error);
    return NextResponse.json({ user: null }); // важно: всегда JSON
  }
}
```

Рисунок 3.4. – Код получения текущего пользователя

– Метод POST в /api/auth/change-password — смена пароля.

Код удаления сессии изображен на рисунке 3.5.

```
export async function POST(request: Request) {
  const session = await getSession();
  if (!session.userId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const { oldPassword, newPassword } = await request.json();
  const user = await prisma.user.findUnique({ where: { id: session.userId } });
  if (!user || !user.passwordHash) {
    return NextResponse.json({ error: 'User not found' }, { status: 404 });
  }

  const isValid = await bcrypt.compare(oldPassword, user.passwordHash);
  if (!isValid) {
    return NextResponse.json({ error: 'Неверный старый пароль' }, { status: 400 });
  }

  const newHash = await bcrypt.hash(newPassword, 10);
  await prisma.user.update({
    where: { id: session.userId },
    data: { passwordHash: newHash },
  });

  return NextResponse.json({ success: true });
}
```

Рисунок 3.5. – Код смены пароля

Контент:

– Методы GET/POST в /api/games — создание игры.

Код создания игры изображен на рисунке 3.6.

```

export async function GET() {
  const games = await prisma.game.findMany({
    include: {
      gameGenres: { include: { genre: true } },
      gamePlatforms: { include: { platform: true } },
      gameFiles: true,
      media: true,
      reviews: { select: { rating: true } },
      subscriptionTiers: { select: { price: true } },
      _count: { select: { reviews: true } },
    },
  });
  return NextResponse.json(games);
}

export async function POST(request: Request) {
  const session = await getSession();
  if (!session.userId || (session.role !== 'developer' && session.role !== 'admin')) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const body = await request.json();
  const { title, description, genreIds, platformIds, mediaUrl, requiredTier } = body;

  if (!title) {
    return NextResponse.json({ error: 'Название обязательно' }, { status: 400 });
  }

  const game = await prisma.game.create({
    data: {
      title,
      description,
      mediaUrl,
      requiredTier: requiredTier || null,
      authorId: session.userId,
      gameGenres: {
        create: genreIds?.map((id: number) => ({ genre: { connect: { id: Number(id) } } })) || [],
      },
      gamePlatforms: {
        create: platformIds?.map((id: number) => ({ platform: { connect: { id: Number(id) } } })) || [],
      },
    },
    include: {
      gameGenres: { include: { genre: true } },
      gamePlatforms: { include: { platform: true } },
      gameFiles: true,
      media: true,
    },
  });
}

```

Рисунок 3.6. – Код создания карточки игры

- Метод GET в /api/articles — статьи и гайды.

Код для получения статей и гайдов изображен на рисунке 3.7.

```
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const gameId = searchParams.get('gameId');
  const where = gameId ? { gameId: Number(gameId) } : {};

  const articles = await prisma.article.findMany({
    where,
    include: {
      game: { select: { title: true } },
      author: { select: { name: true } },
    },
    orderBy: { createdAt: 'desc' },
  });

  return NextResponse.json(articles);
}
```

Рисунок 3.7. – Код для получения статей и гайдов

- Методы GET/POST в /api/news — новостные посты. Код новостных постов изображен на рисунках 3.8. и 3.9.

```

export async function POST(request: Request) {
  const session = await getSession();
  if (!session.userId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  let title: string, content: string, category: string, gameId: number | null;
  let description: string | undefined;
  let mediaUrl: string | undefined;
  let subcategory: string | undefined;

  const contentType = request.headers.get('content-type') || '';
  if (contentType.includes('application/json')) {
    const body = await request.json();
    title = body.title;
    content = body.content;
    category = body.category || 'news';
    gameId = body.gameId !== undefined && body.gameId !== null ? Number(body.gameId) : null;
    description = body.description || undefined;
    mediaUrl = body.mediaUrl || undefined;
    subcategory = body.subcategory || undefined;
  } else {
    const formData = await request.formData();
    title = formData.get('title') as string;
    content = formData.get('content') as string;
    category = (formData.get('category') as string) || 'news';
    gameId = formData.get('gameId') ? Number(formData.get('gameId')) : null;
    description = (formData.get('description') as string) || undefined;
    mediaUrl = (formData.get('mediaUrl') as string) || undefined;
    subcategory = (formData.get('subcategory') as string) || undefined;
  }

  if (!title || !content) {
    return NextResponse.json({ error: 'Заголовок и содержание обязательны' }, { status: 400 });
  }

  const article = await prisma.article.create({
    data: {
      title,
      content,
      category,
      gameId,
      authorId: session.userId,
      description,
      mediaUrl,
      subcategory,
    },
  });
}

```

Рисунок 3.8. – Метод POST для новостных постов


```

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const gameId = searchParams.get('gameId');
  if (!gameId) return NextResponse.json({ error: 'gameId required' }, { status: 400 });

  const session = await getSession();
  const userId = session.userId;

  const newsPosts = await prisma.newsPost.findMany({
    where: { gameId: Number(gameId) },
    include: { author: { select: { name: true } } },
    orderBy: { createdAt: 'desc' },
  });

  // Если пользователь не подписан, скрываем содержимое эксклюзивных новостей
  let subscription = null;
  if (userId) {
    subscription = await prisma.subscription.findFirst({
      where: {
        userId,
        status: 'active',
        tier: { gameId: Number(gameId) },
      },
    });
  }

  const result = newsPosts.map(post => {
    if (post.isExclusive && !subscription) {
      return {
        ...post,
        content: 'Эта новость доступна только подписчикам.',
      };
    }
    return post;
  });

  return NextResponse.json(result);
}

```

Рисунок 3.9. – Метод GET для новостных постов

- Метод GET в /api/comments — комментарии (с поддержкой parentId).

Код комментариев изображен на рисунке 3.10.

```
export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const { id } = await params;
  const comment = await prisma.comment.findUnique({
    where: { id: Number(id) },
    select: { id: true, articleId: true },
  });
  if (!comment) return NextResponse.json({ error: 'Not found' }, { status: 404 });
  return NextResponse.json(comment);
}
```

Рисунок 3.10. – Код комментариев

Бизнес-логика:

- POST /api/subscribe — оформление подписки (с проверкой дубликатов).

Код подписки изображен на рисунке 3.11.

```
export async function POST(request: Request) {
  const session = await getSession();
  if (!session.userId || session.role !== 'developer') {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const { title, content, isExclusive, gameId } = await request.json();
  const post = await prisma.newsPost.create({
    data: {
      title,
      content,
      isExclusive: isExclusive || false,
      gameId: Number(gameId),
      authorId: session.userId,
      mediaUrl,
    },
  });
  return NextResponse.json(post, { status: 201 });
}
```

Рисунок 3.11. – Код подписок

- Методы GET/POST в /api/subscription-tiers — управление tier-ами (Developer).

Код страницы управления подписками изображен на рисунках 3.12. и 3.12.1

```
export async function POST(req: NextRequest) {
  try {
    const session = await getSession();
    if (!session?.userId) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const body = await req.json();
    const { gameId, name, price, benefits, maxSlots } = body;

    const game = await prisma.game.findUnique({
      where: { id: Number(gameId) },
      select: { authorId: true },
    });

    if (!game) {
      return NextResponse.json({ error: 'Game not found' }, { status: 404 });
    }

    const isOwnerOrAdmin = session.userId === game.authorId || session.role === 'admin';
    if (!isOwnerOrAdmin) {
      return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
    }

    const tier = await prisma.subscriptionTier.create({
      data: {
        gameId: Number(gameId),
        name,
        price: Number(price),
        benefits: benefits || '',
        maxSlots: Number(maxSlots) || 100,
      },
    });

    return NextResponse.json(tier, { status: 201 });
  } catch (error) {
    console.error('POST tier error:', error);
    return NextResponse.json({ error: 'Internal error' }, { status: 500 });
  }
}
```

Рисунок 3.12. – Метод POST страницы управления подписками

```

export async function GET(req: NextRequest) {
  const session = await getSession();
  if (!session?.userId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  const { searchParams } = new URL(req.url);
  const gameId = Number(searchParams.get('gameId'));

  if (!gameId) {
    return NextResponse.json({ error: 'gameId required' }, { status: 400 });
  }

  const tiers = await prisma.subscriptionTier.findMany({
    where: { gameId },
    orderBy: { createdAt: 'asc' },
  });

  return NextResponse.json(tiers);
}

```

Рисунок 3.12.1. – Метод GET страницы управления подписками

– Метод POST в /api/tickets — создание тикетов.

Код создания тикета изображен на рисунке 3.13.

```
export async function POST(request: Request) {
  const session = await getSession();
  if (!session.userId) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  let title: string, description: string, type: string, gameId: number;

  const contentType = request.headers.get('content-type');
  if (contentType?.includes('application/json')) {
    const body = await request.json();
    title = body.title;
    description = body.description;
    type = body.type || 'bug';
    gameId = Number(body.gameId);
  } else {
    const formData = await request.formData();
    title = formData.get('title') as string;
    description = formData.get('description') as string;
    type = (formData.get('type') as string) || 'bug';
    gameId = Number(formData.get('gameId'));
  }

  if (!title || !description || !gameId) {
    return NextResponse.json({ error: 'Missing required fields' }, { status: 400 });
  }

  try {
    const ticket = await prisma.ticket.create({
      data: {
        title,
        description,
        type,
        gameId,
        authorId: session.userId,
      },
    });
  }
}
```

Рисунок 3.13. – Код создания тикета

- Метод PATCH в /api/tickets/[id] — обработка тикета (смена статуса, назначение).

Код обработки тикетов изображен на рисунках 3.14. и 3.15.

```
export async function PATCH(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  const session = await getSession();
  if (!session.userId || (session.role !== 'developer' && session.role !== 'admin')) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const { id } = await params;
  const ticketId = Number(id);

  try {
    const body = await request.json();
    const { status, resolution, closeReason } = body;

    const validStatuses = ['open', 'in_progress', 'resolved', 'closed'];
    if (!status || !validStatuses.includes(status)) {
      return NextResponse.json({ error: 'Invalid status' }, { status: 400 });
    }

    // Для закрытия обязательно указать причину
    if (status === 'closed' && !closeReason?.trim()) {
      return NextResponse.json({ error: 'Причина закрытия обязательна' }, { status: 400 });
    }

    // Подготавливаем данные для обновления
    const data: any = {
      status,
      assigneeId: session.userId,
    };
  }
}
```

Рисунок 3.14. – Метод PATCH обработки тикета(1 часть)

```

// Если переводим в решён, сохраняем описание (если передано)
if (status === 'resolved') {
  data.resolution = resolution?.trim() || null;
}

// Если переводим в закрыт, сохраняем причину
if (status === 'closed') {
  data.closeReason = closeReason?.trim();
}

// При возврате из закрытого/решённого в другой статус – сбрасываем соответствующие поля (опционально)
if (status !== 'resolved' && status !== 'closed') {
  data.resolution = null;
  data.closeReason = null;
}

const updatedTicket = await prisma.ticket.update({
  where: { id: ticketId },
  data,
  include: { author: true },
});

```

Рисунок 3.15. – Метод РАСН обработки тикетов(2 часть)

– Метод GET в /api/developer/tickets — тикеты по играм разработчика.

Код отображения “Моих тикетов” изображен на рисунке 3.16.

```

export async function GET() {
  const session = await getSession();
  if (!session.userId || (session.role !== 'developer' && session.role !== 'admin')) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
  }

  const developerGames = await prisma.game.findMany({
    where: { authorId: session.userId },
    select: { id: true },
  });
  const gameIds = developerGames.map(g => g.id);

  const tickets = await prisma.ticket.findMany({
    where: { gameId: { in: gameIds } },
    include: {
      game: { select: { title: true } },
      author: { select: { name: true } },
    },
    orderBy: { createdAt: 'desc' },
  });

  return NextResponse.json(tickets);
}

```

Рисунок 3.16. – Код отображения тикетов разработчика

– Метод GET в /api/messages/conversation/[id] — чат.

Код сообщений изображен на рисунке 3.17.

```
export async function GET(  
  request: Request,  
  { params }: { params: Promise<{ id: string }> }  
) {  
  const session = await getSession();  
  if (!session.userId) {  
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });  
  }  
  
  const { id } = await params;  
  const conversationId = Number(id);  
  
  const conv = await prisma.conversation.findUnique({  
    where: { id: conversationId },  
    select: { user1Id: true, user2Id: true },  
  });  
  if (!conv || (conv.user1Id !== session.userId && conv.user2Id !== session.userId)) {  
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 });  
  }  
  
  // Помечаем все сообщения собеседника как прочитанные  
  await prisma.message.updateMany({  
    where: {  
      conversationId,  
      senderId: { not: session.userId },  
      isRead: false,  
    },  
    data: { isRead: true },  
  });  
  
  const messages = await prisma.message.findMany({  
    where: { conversationId },  
    include: {  
      sender: { select: { id: true, name: true, avatarUrl: true } },  
      media: true,  
    },  
    orderBy: { createdAt: 'asc' },  
  });  
  
  return NextResponse.json(messages);  
}
```

Рисунок 3.17. – Код чата

– Метод POST в /api/upload — загрузка медиа-файлов.

Код загрузки медиа-файлов изображен на рисунке 3.18.

```
export async function POST(request: Request) {
  try {
    const formData = await request.formData();
    const file = formData.get('file') as File | null;

    if (!file) {
      return NextResponse.json({ error: 'Файл не выбран' }, { status: 400 });
    }

    const allowedTypes = ['image/jpeg', 'image/png', 'image/gif', 'image/webp', 'video/mp4', 'video/webm'];
    if (!allowedTypes.includes(file.type)) {
      return NextResponse.json({ error: 'Неподдерживаемый тип файла' }, { status: 400 });
    }

    const maxSize = 10 * 1024 * 1024; // 10 МБ
    if (file.size > maxSize) {
      return NextResponse.json({ error: 'Файл слишком большой (макс. 10 МБ)' }, { status: 400 });
    }

    const timestamp = Date.now();
    const ext = file.name.split('.').pop() || 'bin';
    const fileName = `${timestamp}-${Math.random().toString(36).substring(2)}${ext}`;

    const uploadDir = path.join(process.cwd(), 'public', 'uploads');
    await mkdir(uploadDir, { recursive: true });

    const buffer = Buffer.from(await file.arrayBuffer());
    await writeFile(path.join(uploadDir, fileName), buffer);

    return NextResponse.json({ url: `/uploads/${fileName}` });
  } catch (error: any) {
    console.error('Ошибка загрузки файла:', error.message);
    return NextResponse.json(
      { error: 'Внутренняя ошибка сервера при загрузке файла' },
      { status: 500 }
    );
  }
}
```

Рисунок 3.18. – Код загрузки медиа-файлов

Социальное:

- Метод POST в /api/user/follow — подписка на пользователя.

Код подписки на пользователя изображен на рисунке 3.19.

```
export async function POST(request: Request) {
  try {
    const session = await getSession();
    if (!session.userId) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
    }

    const { targetUserId } = await request.json();
    if (!targetUserId || targetUserId === session.userId) {
      return NextResponse.json({ error: 'Недопустимый пользователь' }, { status: 400 });
    }

    const existing = await prisma.follow.findUnique({
      where: {
        followerId_followingId: {
          followerId: session.userId,
          followingId: targetUserId,
        },
      },
    });

    if (existing) {
      await prisma.follow.delete({ where: { id: existing.id } });
      return NextResponse.json({ following: false });
    } else {
      await prisma.follow.create({
        data: {
          followerId: session.userId,
          followingId: targetUserId,
        },
      });
      return NextResponse.json({ following: true });
    }
  } catch (error: any) {
    console.error('Follow API error:', error);
    return NextResponse.json({ error: 'Ошибка сервера' }, { status: 500 });
  }
}
```

Рисунок 3.19. – Код подписки на пользователя

- Метод GET в /api/notifications — уведомления.

Код уведомлений изображен на рисунке 3.20

```
export async function GET(request: Request) {
  const session = await getSession();
  if (!session.userId) return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });

  const { searchParams } = new URL(request.url);
  const showAll = searchParams.get('all') === 'true';

  const where: any = { userId: session.userId };
  if (!showAll) where.isRead = false;

  const notifications = await prisma.notification.findMany({
    where,
    orderBy: { createdAt: 'desc' },
    take: 50,
  });

  return NextResponse.json(notifications);
}
```

Рисунок 3.20. – Код уведомлений

Middleware и безопасность

- src/middleware.ts — единая точка входа для всех HTTP-запросов.

Код обработчика запросов изображен на рисунке 3.21.

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const response = NextResponse.next();
  response.headers.set('x-pathname', request.nextUrl.pathname);
  return response;
}
```

Рисунок 3.21. – Единая точка входа для всех HTTP-запросов

Инъекция заголовка x-pathname позволяет клиентским компонентам определять текущий путь без useRouter() (недоступного в серверных компонентах App Router).

Ролевая защита реализована на уровне каждого API-роута:

```

    if (!session.userId || (session.role !== 'developer' && session.role !==
'admin')) {
        return NextResponse.json({ error: 'Forbidden' }, { status: 403 });
    }

```

3.3. Проектирование клиентской части (Frontend)

App Router структура

src/app/

```

├── page.tsx          # Главная / каталог игр
├── layout.tsx (привязан к quick-menu)  # Корневой layout с навигацией
├── globals.css       # Глобальные стили
├── admin/            # Админ-панель
├── api/              # API Routes
├── developer/
│   ├── dashboard/   # Панель разработчика
│   ├── tickets/     # Тикеты по играм
│   └── subscriptions/ # Управление tier-ами
├── games/[id]/       # Карточка игры
│   ├── edit/        # Редактирование
│   ├── news/        # Новости игры
│   └── subscribe/    # Подписка
├── guides/           # База знаний
├── knowledge/        # Статьи
├── login/            # Авторизация
├── messages/         # Личные сообщения
├── news/             # Новостная лента
├── profile/          # Личный кабинет
├── register/         # Регистрация
└── user/[id]/        # Публичный профиль

```

Компонентная архитектура

CRUD-компоненты:

- CreateGameForm
- CreateArticleForm
- CreateNewsForm
- CreateTicketForm
- EditGameButton
- DeleteGameButton

Медиа:

- CoverUploader
- MediaUploader
- GameGallery — загрузка и отображение медиа-контента.

Подписки:

- SubscribeButton
- TierSelect
- DeveloperSubscriptions — UI оформления и управления подписками.

Социальное:

- FollowButton
- CommentSection
- GameCommentSection
- GameReviewsSection
- SendMessageButton
- ExistingChatButton.

Тикеты:

- TicketSection
- DeveloperTickets
- QuickStats

Навигация и фильтры:

- Navigation
- GenreSelect
- GenreMultiSelect
- PlatformSelect.

					ДП.09.02.07.25.18 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		33

4. Функциональное тестирование, отладка и оптимизация работы системы.

Проверка корректности работы ключевых модулей: аутентификации, подписок, тикетов, комментариев, чата, загрузки файлов, ролевой защиты.

4.1. Тестовое окружение

Таблица 4.1. – Тестовое окружение

Параметр	Значение
ОС	Windows 11
Браузер	Google Chrome 124
Сервер	Next.js 15 dev (localhost:3000)
База данных	SQLite (dev.db)
ORM	Prisma Studio (localhost:5555)

4.2. Тест-кейсы и результаты

Таблица 4.2 - ТК Регистрации

Шаг	Действие	Результат
1	Открыть /register	Форма отображена
2	Заполнить поля, нажать "Зарегистрироваться"	Редирект на /login
3	Проверить БД	Запись в User с role="user"

Таблица 4.3. - ТК Аутентификации

Шаг	Действие	Результат
1	Войти с корректными данными	Сессия создана, редирект
2	Проверить /api/auth/me	Объект пользователя возвращён

Таблица 4.4. - ТК Создания игры (Developer)

Шаг	Действие	Результат
1	Войти как Developer	Успешно
2	Создать игру через CreateGameForm	Игра сохранена, обложка загружена

Таблица 4.5. - ТК Подписки на игру(User)

Шаг	Действие	Результат
1	Выбрать tier, нажать "Подписаться"	Подписка создана
2	Повторить попытку	Ошибка 400 "Already subscribed"

Таблица 4.6. - ТК Обработки тикетов

Шаг	Действие	Результат
1	User создаёт тикет	Статус "Открыт"
2	Developer меняет статус	"В процессе", уведомление отправлено
3	Закреть без причины	Ошибка 400

Таблица 4.7. - ТК Отправки комментария

Шаг	Действие	Результат
1	Написать комментарий	Отображается
2	Ответить на комментарий	Вложенность depth=1

Таблица 4.8. - ТК Чата

Шаг	Действие	Результат
1	Отправить сообщение	Диалог создан
2	Удалить диалог для себя	Скрыт для User1, видим для User2
3	User2 тоже удаляет	Физическое удаление

Таблица 4.9. - ТК Загрузки файлов

Шаг	Действие	Результат
1	Загрузить .jpg (5 МБ)	Успешно
2	Загрузить .exe	Ошибка 400 (тип)
3	Загрузить .png (15 МБ)	Ошибка 400 (размер)

Таблица 4.10. - ТК Ролевой защиты

Шаг	Действие	Результат
1	User делает POST /api/developer/games	403 Forbidden
2	Guest делает POST /api/subscribe	401 Unauthorized

4.3. Заключение по тестированию

Все 9 тест-кейсов пройдены успешно. Система корректно обрабатывает граничные случаи: повторные подписки, закрытие тикетов без обязательных полей, мягкое удаление диалогов, валидацию файлов. Ролевая модель надёжно защищает API-эндпоинты.

5 Расчет себестоимости разработки платформы.

Себестоимость – это денежное выражение текущих затрат предприятия на производство и реализацию продукции (работ, услуг).

Полная себестоимость работ по разработке платформы для продвижения игр C , руб., определяется по формуле (1).

$$C = C_m + C_z + C_c + C_a + C_{эл} + C_{пр}, \quad (1)$$

где C_m – материальные затраты, руб.;

C_z – затраты на оплату труда работников, руб.;

C_c – отчисления на социальные нужды, руб.;

C_a – амортизационные отчисления, руб.;

$C_{эл}$ – затраты на электроэнергию, руб.;

$C_{пр}$ – прочие расходы, руб.

Материальные затраты – это суммы затрат на основные материалы и на вспомогательные материалы. Основные и вспомогательные материалы при проектировании и разработке платформы для продвижения игр не использовались. Расчёт материальных затрат не производится.

Затраты на оплату труда – это вознаграждение за работу сотрудников в зависимости от их профессионального уровня, условий и сложности работы.

Затраты на оплату труда работников C_z , руб., определяются по формуле (2).

$$C_z = C_{осн} \cdot K_{пр} \cdot K_{доп} \cdot K_{рк}, \quad (2)$$

где C_z – основная заработная плата рабочих, руб.;

$K_{пр}$ – коэффициент, учитывающий размер премии. $K_{пр}$ принимается 1,5.

$K_{доп}$ – коэффициент, учитывающий размер дополнительной заработной платы. $K_{доп}$ принимается 1,25.

					ДП.09.02.07.25.18 ПЗ	Лист
						37
Изм.	Лист	№ докум.	Подпись	Дата		

$K_{рк}$ – районный коэффициент, учитывающий климатические условия труда. Районный коэффициент, утверждённый по Кемеровской области $K_{рк}=1,3$.

Основная заработная плата $C_{осн}$, руб, определяется по формуле (3).

$$C_{осн} = T_i \cdot t_i, \quad (3)$$

где T_i – часовая тарифная ставка, руб.;

t_i – трудоемкость работ, ч.

Принимаем работника 11 разряда $T_i = 114,15$ руб. Данные взяты из Единой тарифной сетки.

Затраты рабочего времени, необходимые для выполнения работы установленного объёма, измеряются в часах или нормо-часах и называются трудоемкостью работы. Расчёт трудоемкости работ приведён в таблице 5.1.

Таблица 5.1. – Расчет трудоемкости работ

Номер и наименование операции	Трудоемкость работы, ч.
1. Подбор информации	30
2. Настройка сервера	15
3. Разработка объектов базы данных	25
4. Разработка пользовательского интерфейса	111
5. Разработка бизнес-логики	112
6. Разработка мобильной версии для сайта	10
7. Тестирование и отладка программных продуктов	50
Итого	353

Основная заработная плата $C_{осн}$, руб, определена по формуле (3).

$$C_{осн} = 114,15 \cdot 353 = 40924,95 \text{ руб}$$

Затраты на оплату труда работников C_3 , руб., определяется по формуле (2).

$$C_3 = 40924,95 \times 1,5 \times 1,25 \times 1,3 = 99\,754,56 \text{ руб}$$

Отчисления на социальные нужды – это один из элементов себестоимости продукции (работ, услуг), в котором отражаются обязательные

отчисления по установленным законодательством нормам государственного социального страхования в Фонд социального страхования Российской Федерации, Пенсионный фонд Российской Федерации, Государственный фонд занятости населения Российской Федерации и фонды обязательного медицинского страхования.

В 2023 году из-за объединения ПФР и ФСС и введения единого налогового платежа взносы на ОПС, ОМС и ВНиМ объединили в один платеж: теперь единый круг застрахованных лиц, единая база для начисления и единый тариф. Стандартные ставки страховых взносов:

1. На обязательное пенсионное, медицинское и социальное страхование на случай временной нетрудоспособности и материнства — 30%.

2. На травматизм – от 0,2% до 8,5% в зависимости от класса профессионального риска, присвоенного основному осуществляемому виду деятельности. Вероятность ущерба здоровью, возникающая в результате исполнения трудовых обязанностей, называется профессиональным риском. Чем выше профессиональный риск, тем выше размер ставки страхового взноса. Принимаем 0,2%.

Отчисления на социальные нужды C_c , руб., рассчитываются по формуле (4).

$$C_c = C_z * \frac{P_c}{100}, \quad (4)$$

где C_z - затраты на оплату труда работников, руб.;

P_c – процент отчислений на социальные нужды, %. Принимается 30,2%.

$$C_c = 99\,754,56 * \frac{30,2}{100} = 30\,132,21 \text{ руб}$$

Амортизация основных фондов является важным аспектом финансовой деятельности компаний. Она позволяет учитывать износ основных средств, которые необходимы для производственной деятельности. Установка норм амортизации по группам основных средств позволяет правильно распределить расходы на обновление и покупку нового оборудования.

					ДП.09.02.07.25.18 ПЗ	Лист
						39
Изм.	Лист	№ докум.	Подпись	Дата		

Портативным компьютерам (ноутбукам, планшета, смартфонам) в Общероссийском классификаторе выделен отдельный код 320.26.20.11. Такая техника, как и обычные компьютеры, относится ко второй амортизационной группе со сроком полезного использования от двух до трех лет.

Сумма амортизационных отчислений А, руб., определяется по формуле (5).

$$A = C_{\text{п}} \cdot N_{\text{а}} / 100 \%, \quad (5)$$

где $C_{\text{п}}$ – полная первоначальная стоимость основных средств, руб.;

$N_{\text{а}}$ - норма амортизации, %. Принимаем 17%.

Расчёт полной первоначальной стоимости применяемого оборудования приведён в таблице 5.2.

Таблица 5.2.– Расчёт полной первоначальной стоимости оборудования

Наименование оборудования	Количество	Цена, руб.	Полная первоначальная стоимость, руб.
Системный блок	1	10000	10000
Монитор	1	1000	1000
Клавиатура	1	1000	1000
Мышь	1	1000	1000
Видеокарта	1	30000	30000
Плашки видео-памяти	2	7500	15000
Блок питания	1	2000	2000
Материнская плата	1	23000	23000
Кулер	1	2500	2500
Итого			85500

$$A = 85500 \cdot 17\% / 100\% = 14535 \text{ руб}$$

Затраты на потребляемую электроэнергию $C_{\text{эл}}$, руб., определяются по формуле (6).

$$C_{\text{эл}} = t_i \cdot P \cdot \Pi_{\text{эл}}, \quad (6)$$

где t_i – время работы электрооборудования, час.;

P – потребляемая мощность устройства по паспорту компьютера, $P = 0,400$ кВт;

$\Pi_{эл}$ – цена за 1 кВт/час электроэнергии, $\Pi_{эл} = 3,76$ руб.

$$C_{эл} = 353 \times 0,400 \times 3,76 = 530,91 \text{ руб}$$

Прочие расходы представляют собой расходы на оплату транспортных услуг, оплату канцтоваров, командировок и т.д. При разработке платформы для продвижения игр затраты на прочие расходы отсутствуют.

Полная себестоимость работ по разработке платформы C , руб., определяется по формуле (1).

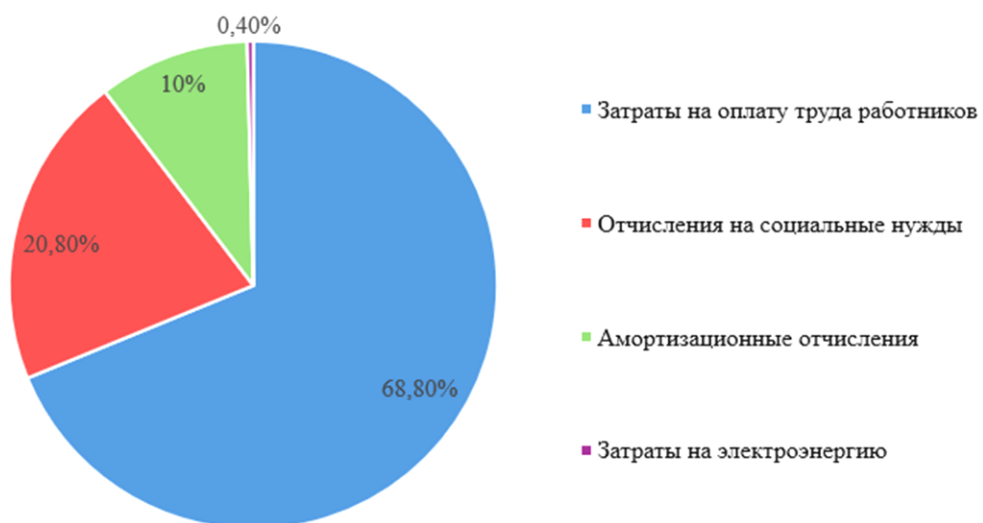
Результаты расчётов себестоимости оформлены в виде таблицы 5.3.

Таблица 5.3 – Расчёт затрат на выполнение работ

Наименование затрат	Затраты, руб.	Доля затрат, %
Материальные затраты	-	-
Затраты на оплату труда работников	99 754,56	68,8
Отчисления на социальные нужды	30 132,21	20,8
Амортизационные отчисления	14535	10
Затраты на электроэнергию	530,91	0,4
Прочие расходы	-	-
Итого затрат	144 952,68	100,0

На основании таблицы построен график структуры затрат на разработку веб-сайта для чтения манги, показан на рисунке 5.1.

Структура затрат на разработку платформы для продвижения игр



Большую долю в общей сумме затрат на выполнение работ составляют затраты на оплату труда работников. Это связано с тем, что для разработки веб-сайта требуется высокая квалификация работника, который выполняет разработку.

Заключение

В ходе выполнения проекта разработана полнофункциональная веб-платформа для продвижения и поддержки видеоигр. Реализованы все запланированные модули:

Аутентификация и авторизация на базе iron-session с тремя ролями пользователей.

- Каталог игр с фильтрацией, поиском, медиа-галереей и системой оценок.
- Монетизация через tier-based подписки с защитой от повторного оформления.
- Техподдержка с полным жизненным циклом тикетов, статусной машиной и уведомлениями.
- Контент — новости, гайды, статьи с рекурсивными комментариями.
- Коммуникация — личные сообщения с мягким удалением диалогов и подписки на разработчиков.
- Администрирование — управление пользователями, ролями и модерация.

Технологический стек — Next.js 15, Prisma 5.22, TypeScript, TailwindCSS, iron-session, SQLite — продемонстрировал свою эффективность для быстрого прототипирования и дальнейшего масштабирования.

Перспективы развития:

- Интеграция платёжных систем (Stripe, ЮKassa) для реальных транзакций.
- Миграция на PostgreSQL и Redis для production-нагрузок.
- Real-time уведомления через WebSockets.
- Мобильное приложение на React Native.
- Система аналитики и дашбордов для разработчиков.
- Система аналитики и дашбордов для разработчиков.

					ДП.09.02.07.25.18 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		43

Список источников

1. Панфилов, К. С. Создание веб-сайта от замысла до реализации : практическое руководство / К. С. Панфилов. - 2-е изд. - Москва : ДМК Пресс, 2023. - 438 с. - ISBN 978-5-89818-476-6. - Текст : электронный. - URL: <https://znanium.com/catalog/product/2106246> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

2. Болдырев, М. В. Платформенная революция : вызов традиционной экономике : монография / М. В. Болдырев. – Москва : Издательско-торговая корпорация «Дашков и К°», 2026. - 212 с. – ISBN 978-5-394-06344-2. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2242864> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

3. Основы работы с CSS : краткий курс / . - Москва : ИНТУИТ, 2016. - 148 с. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2152337> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

4. Хэррон, Д. Node.js. Разработка серверных веб-приложений на JavaScript : практическое руководство / Д. Хэррон ; пер. с англ. А. А. Слинкина. - 2-е изд. - Москва : ДМК Пресс, 2023. - 145 с. - ISBN 978-5-89818-632-6. - Текст : электронный. - URL: <https://znanium.com/catalog/product/2108525> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

5. Сухов, К. К. HTML5 — путеводитель по технологии : практическое руководство / К. К. Сухов. - 2-е изд. - Москва : ДМК Пресс, 2023. - 353 с. - ISBN 978-5-89818-532-9. - Текст : электронный. - URL: <https://znanium.com/catalog/product/2107209> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

6. Вагин, Д. В. Современные технологии разработки веб-приложений : учебное пособие / Д. В. Вагин, Р. В. Петров. - Новосибирск : Изд-во НГТУ, 2019. - 52 с. - ISBN 978-5-7782-3939-5. - Текст : электронный. - URL: <https://znanium.com/catalog/product/1866926> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

					ДП.09.02.07.25.18 ПЗ	Лист
						44
Изм.	Лист	№ докум.	Подпись	Дата		

7. Малашкевич, В. Б. Интернет-программирование : лабораторный практикум / В. Б. Малашкевич. - Йошкар-Ола : Поволжский государственный технологический университет, 2017. - 96 с. - ISBN 978-5-8158-1854-5. - Текст : электронный. - URL: <https://znanium.com/catalog/product/1873435> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

8. Сычев, А. В. Web-технологии : краткий учебный курс / А. В. Сычев. - Москва : ИНТУИТ, 2016. - 304 с. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2137134> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

9. Сакулин, С. А. Основы интернет-технологий: HTML, CSS, JavaScript, XML : учебное пособие / С. А. Сакулин. - Москва : Издательство МГТУ им. Баумана, 2017. - 112 с. - ISBN 978-5-7038-4724-4. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2169329> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

10. Рогов, Е.В. PostgreSQL 16 изнутри : практическое руководство / Е. В. Рогов. – Москва : ДМК Пресс, 2024. - 666 с. – ISBN 978-5-93700-305-8. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2205083> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

11. Стасышин, В. М. Работа с базами данных : учебное пособие / В. М. Стасышин, Т. Л. Стасышина, М. А. Сивак. - Новосибирск : Изд-во НГТУ, 2025. - 76 с. - ISBN 978-5-7782-5472-5. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2244401> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

12. Организация баз данных и систем управления ими : методическое пособие / сост. Т. В. Соколова. - 2-е изд., стер. - Москва : ФЛИНТА, 2024. - 37 с. - ISBN 978-5-9765-5658-4. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2179295> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

13. Технологии программирования. Работа с базами данных : методическое пособие / Д. И. Попов, С. С. Валеев, Е. Д. Попова, Т. Л. Салова.

					ДП.09.02.07.25.18 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		45

- 2-е изд., стер. - Москва : ФЛИНТА, 2024. - 40 с. - ISBN 978-5-9765-5645-4. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2179291> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

14. Бедердинова, О. И. Создание приложений баз данных в среде Visual Studio : учебное пособие / О.И. Бедердинова, Т.А. Минеева, Ю.А. Водовозова. — Москва : ИНФРА-М, 2021. — 94 с. - ISBN 978-5-16-109411-2. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/1243816> (дата обращения: 03.06.2026)

15. Жуков, Р. А. Базы данных : учебно-методическое пособие по дисциплине «Базы данных» для направления подготовки 38.03.05 «Бизнес-информатика» (бакалавриат) / Р. А. Жуков. - Москва ; Берлин : Директ-Медиа, 2019. - 176 с. - ISBN 978-5-4499-0225-2. - Текст : электронный. - URL: <https://znanium.com/catalog/product/1874923> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

16. Экономика : учебное пособие / С.Д. Резник, З.А. Мебадури, Е.В. Духанина, Т.Н. Чудайкина ; под общ. ред. д-ра экон. наук, проф. С.Д. Резника. — 4-е изд., перераб. и доп. — Москва : ИНФРА-М, 2026. — 242 с. — (Высшее образование). — DOI 10.12737/2227252. - ISBN 978-5-16-021494-8. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2227252> (дата обращения: 31.05.2026). – Режим доступа: по подписке.

17. Полищук, Ю. В. Базы данных и их безопасность : учебное пособие / Ю.В. Полищук, А.С. Боровский. — Москва : ИНФРА-М, 2026. — 210 с. — (Среднее профессиональное образование). - ISBN 978-5-16-016151-8. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2212079> (дата обращения: 31.05.2026). – Режим доступа: по подписке.

18. Федотов, В. А. Экономика : учебник / В.А. Федотов, О.В. Комарова. — 4-е изд., перераб. и доп. — Москва : ИНФРА-М, 2026. — 196 с. — (Среднее профессиональное образование). - ISBN 978-5-16-015038-3. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2243312> (дата обращения: 31.05.2026). – Режим доступа: по подписке.

					ДП.09.02.07.25.18 ПЗ	Лист
						46
Изм.	Лист	№ докум.	Подпись	Дата		

19. Сычев, А. В. Перспективные технологии и языки веб-разработки : краткий курс / А. В. Сычев. - Москва : ИНТУИТ, 2016. - 367 с. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2155112> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

20. Исаченко, О. В. Программное обеспечение компьютерных сетей : учебное пособие / О.В. Исаченко. — 2-е изд., испр. и доп. — Москва : ИНФРА-М, 2026. — 158 с. — (Среднее профессиональное образование). - ISBN 978-5-16-015447-3. - Текст : электронный. - URL: <https://znanium.ru/catalog/product/2201207> (дата обращения: 03.06.2026). – Режим доступа: по подписке.

					ДП.09.02.07.25.18 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		47